

Resenha 10: Design by Contract — A Estabilidade através do Rigor

Autor da Resenha: Bernardo Gonçalves Levy **Disciplina:** Projeto de Software

Artigo Analisado: Design by Contract (Bertrand Meyer, 1991)

1. Visão Geral: O Software como um Acordo Formal

Neste artigo seminal, Bertrand Meyer apresenta o conceito de **Design by Contract (DbC)**, utilizando a metáfora de contratos comerciais para guiar a construção de software confiável. A ideia central é que a interação entre diferentes unidades de software (rotinas e seus clientes) deve ser regida por contratos documentados que explicitam obrigações e benefícios mútuos. Através de **asserções** — como pré-condições, pós-condições e invariantes de classe — o desenvolvedor define as regras de engajamento que garantem que o sistema opere em um estado consistente.

2. Análise Crítica: A Rejeição da Programação Defensiva

Meyer propõe uma mudança de paradigma ao se posicionar contra a "programação defensiva" tradicional. Ele argumenta que o excesso de verificações redundantes tanto no cliente quanto no fornecedor aumenta a complexidade e diminui a confiabilidade do sistema. No DbC, as responsabilidades são divididas de forma cirúrgica:

- **Pré-condições:** Representam as obrigações do cliente; se não forem atendidas, a rotina não é obrigada a processar nada.
- **Pós-condições:** São as garantias do fornecedor; se a pré-condição foi cumprida, a rotina deve entregar o resultado prometido.
- **Invariantes de Classe:** Propriedades que devem permanecer verdadeiras em todos os estados "observáveis" do objeto.

O autor também introduz uma visão rigorosa sobre exceções: elas só devem ocorrer quando um contrato falha. A resposta a uma falha deve ser o pânico organizado (notificar o cliente) ou a retomada (*retry*) após estabilizar o estado do objeto (restaurar o invariante).

3. Aplicação Prática: Contratos na Orquestração de Pesquisas

Na minha rotina na **Offerwise**, onde gerencio lógicas complexas de questionários que dependem de integrações via APIs e manipulação de DOM, os princípios de Meyer são transformadores:

- **Contratos em APIs de Terceiros:** Quando programo lógicas que consomem dados externos, trato as pré-condições como validadores essenciais de entrada. Se a API de um cliente não entrega o formato esperado (quebra do contrato), o DbC ensina

que a responsabilidade não é do meu script tentar "adivinhar" o dado, mas sim sinalizar a falha imediatamente para evitar corrupção de dados na base.

- **Invariantes de Sessão:** Em questionários dinâmicos com jQuery, estabeleço invariantes mentais (e às vezes via código) para garantir que certas variáveis globais de progresso nunca sejam negativas ou superiores ao total de perguntas. O DbC reforça que qualquer manipulação de tela deve, ao final, restaurar a consistência dessa lógica central.
- **Garantia de Entrega (Pós-condições):** Ao desenvolver funções de "limpeza" de respostas antes do envio final, aplico o conceito de assegurar que o estado final do objeto de dados esteja exatamente conforme o dicionário da pesquisa, eliminando a necessidade de verificações posteriores exaustivas no servidor.

4. Conclusão

O artigo de Meyer demonstra que a confiabilidade não nasce do medo (programação defensiva), mas da clareza. Ao aceitar que rotinas podem ser parciais e que contratos devem ser honrados, criamos sistemas mais simples e, conseqüentemente, mais robustos. Para um desenvolvedor lidando com a volatilidade da pesquisa de mercado, o DbC oferece a fundação necessária para construir scripts que sobrevivem à complexidade sem se tornarem "bolas de lama" incontroláveis.